

Chapter 10 — Binary Search on Answer

0. What You Will Learn in This Chapter

Binary Search on Answer is one of the most powerful interview patterns. It is used when the array itself may not be sorted, but the **answer space** has a monotonic property.

Instead of searching inside an array, we search over possible answers.

This chapter is very important for medium and hard problems.

By the end of this chapter, you should understand:

- What Binary Search on Answer means
- How it differs from normal binary search
- What monotonic feasibility means
- How to define the search range
- How to write a feasibility function
- How to minimize a maximum value
- How to maximize a minimum value
- How to avoid overflow
- How to handle impossible cases
- How to choose left and right
- How to explain this pattern in interviews

Core LeetCode problems covered in this chapter:

- 1.1. Koko Eating Bananas
 - 2.2. Capacity To Ship Packages Within D Days
 - 3.3. Find the Smallest Divisor Given a Threshold
 - 4.4. Minimum Number of Days to Make m Bouquets
 - 5.5. Split Array Largest Sum
 - 6.6. Magnetic Force Between Two Balls
 - 7.7. Maximum Candies Allocated to K Children
 - 8.8. Minimum Time to Complete Trips
 - 9.9. Minimum Speed to Arrive on Time
 - 10.10. Minimum Limit of Balls in a Bag
 - 11.11. Minimize Max Distance to Gas Station
 - 12.12. Kth Smallest Number in Multiplication Table
 - 13.13. Find K-th Smallest Pair Distance
 - 14.14. Kth Smallest Element in a Sorted Matrix
 - 15.15. Find the Duplicate Number, value-space binary search
-

1. What Is Binary Search on Answer?

Normal binary search searches inside a sorted array.

Example:

```
nums = {1, 3, 5, 7, 9}
target = 7
```

Binary Search on Answer searches over possible answer values.

Example:

Koko Eating Bananas:

piles = {3, 6, 7, 11}

h = 8

Question:

What is the minimum eating speed K so Koko can finish within h hours?

Possible speeds:

1, 2, 3, 4, 5, 6, ..., 11

If speed 4 works, then speed 5, 6, 7, ... also work.

So feasibility looks like:

false false false true true true true

We binary search for the first true.

2. Why Binary Search on Answer Exists

Many problems ask for:

- minimum possible maximum
- maximum possible minimum
- minimum speed
- minimum capacity
- minimum time
- smallest divisor
- largest minimum distance

Brute force tries every answer.

Example:

Koko speed from 1 to max pile.

for speed from 1 to maxPile:

if canFinish(speed):

return speed

If maxPile is huge, this is too slow.

Binary search reduces answer search from:

$O(\text{answerRange})$

to:

$O(\log \text{answerRange})$

Each guess is checked with a feasibility function.

3. Core Idea

Binary Search on Answer has three steps:

Step 1 — Define Search Space

Find minimum and maximum possible answer.

Example:

Koko:

```
left = 1  
right = maxPile
```

Shipping packages:

```
left = maxWeight  
right = totalWeight
```

Magnetic force:

```
left = 1  
right = maxPosition - minPosition
```

Step 2 — Define Feasibility Function

A function that answers:

```
Can this guessed answer work?
```

Example:

```
bool canFinish(int speed)
```

or:

```
bool canShip(int capacity)
```

or:

```
bool canPlace(int distance)
```

Step 3 — Use Monotonicity

If searching for minimum feasible answer:

```
false false false true true true
```

Use first true binary search.

If searching for maximum feasible answer:

```
true true true false false false
```

Use last true binary search.

4. Monotonic Feasibility

This is the heart of the pattern.

A condition is monotonic if once it becomes true, it stays true, or once it becomes false, it stays false.

4.1 Minimum Feasible Pattern

Example:
Minimum speed.
If speed 5 works, speed 6 also works.
Pattern:

speed: 1 2 3 4 5 6 7
works: F F F T T T T

Find first true.

4.2 Maximum Feasible Pattern

Example:
Maximum minimum magnetic force.
If distance 5 works, distance 4 also works.
Pattern:

distance: 1 2 3 4 5 6 7
works: T T T T T F F

Find last true.

5. When to Use Binary Search on Answer

Use this pattern when the problem asks:

5.1 Minimize a Maximum

Examples:

- Split Array Largest Sum
- Capacity To Ship Packages
- Minimized Maximum of Products
- Minimum Limit of Balls in a Bag

Common phrase:

minimize the largest...
minimum possible maximum...

5.2 Maximize a Minimum

Examples:

- Magnetic Force Between Two Balls
- Aggressive Cows
- Maximize minimum distance

Common phrase:

maximize the minimum...
largest minimum...

5.3 Minimum Speed / Rate / Capacity / Time

Examples:

- Koko Eating Bananas
- Minimum Speed to Arrive on Time
- Minimum Time to Complete Trips
- Smallest Divisor

5.4 K-th Smallest / Counting Less Than or Equal

Examples:

- Kth Smallest Number in Multiplication Table
- Kth Smallest Pair Distance
- Kth Smallest Element in Sorted Matrix

Common idea:

How many values are $\leq$ mid?

If count is at least k, answer is $\leq$ mid.

5.5 Answer Range Is Numeric and Large

If answer can be from 1 to 10^9 , brute force is impossible.

Binary search answer space.

6. When NOT to Use Binary Search on Answer

6.1 Feasibility Is Not Monotonic

If guesses work like:

false true false true

binary search is invalid.

6.2 You Cannot Check a Guess Efficiently

If feasibility check is as hard as solving original problem, binary search may not help.

6.3 Direct Formula Exists

If answer can be computed directly, do not binary search.

Example:

$\text{ceil}(\text{total} / k)$

if no constraints complicate it.

6.4 Need Exact Index in an Existing Sorted Array

Use normal binary search.

7. Mental Models

7.1 Guess the Answer

Binary Search on Answer is:

I do not know the answer, so I guess one possible answer and ask if it works.

7.2 Feasibility Wall

Imagine a wall between impossible and possible answers.

For minimum feasible:

impossible impossible impossible possible possible possible

Binary search finds the wall.

7.3 Budget Model

Many problems ask whether a guess fits within a budget:

- hours
- days
- capacity
- threshold
- operations
- trips
- bouquets

If guess fits budget, try smaller.

If not, try larger.

7.4 Greedy Feasibility

The feasibility function is often greedy.

Example:

Shipping packages:

- fill current day until capacity exceeded
- then start new day

This tells how many days are needed for a capacity.

8. Pattern Recognition Checklist

Ask:

2. Is the answer numeric?
3. Is there a minimum or maximum answer?
4. Can I guess an answer?
5. Can I check if the guess works?
6. If a guess works, do larger guesses also work?
7. If a guess works, do smaller guesses also work?
8. Is the answer range large?
9. Does the problem say minimize maximum?
10. Does the problem say maximize minimum?
11. Does the problem involve speed, time, capacity, distance, divisor, threshold, or K-th smallest?

If yes, binary search on answer may apply.

9. C++ Tools Needed

9.1 Safe Mid

```
long long mid = left + (right - left) / 2;
```

Use long long if range is large.

9.2 Ceiling Division

Very common.

For positive integers:

```
ceil(a / b)
```

as integer:

```
(a + b - 1) / b
```

Example:

```
hours += (pile + speed - 1) / speed;
```

Use long long when needed:

```
hours += (pile + speed - 1LL) / speed;
```

9.3 Accumulate Sum

```
long long total = accumulate(nums.begin(), nums.end(), 0LL);
```

Use 0LL to force long long.

9.4 Max Element

```
int maxVal = *max_element(nums.begin(), nums.end());
```

9.5 Min Element

```
int minVal = *min_element(nums.begin(), nums.end());
```

10. Core Templates

Template 1 — Minimum Feasible Answer

For pattern:

```
false false false true true true
```

Find first true.

```
int binarySearchMinAnswer(int left, int right) {  
    int answer = right;
```

```
while (left <= right) {
    int mid = left + (right - left) / 2;

    if (can(mid)) {
        answer = mid;
        right = mid - 1;
    } else {
        left = mid + 1;
    }
}

return answer;
}
```

Alternative clean version:

```
int binarySearchMinAnswer(int left, int right) {
    while (left < right) {
        int mid = left + (right - left) / 2;

        if (can(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}
```

Template 2 — Maximum Feasible Answer

For pattern:

true true true false false false

Find last true.

```
int binarySearchMaxAnswer(int left, int right) {
    int answer = left;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (can(mid)) {
            answer = mid;
            left = mid + 1;
        }
    }

    return answer;
}
```

```
    } else {  
        right = mid - 1;  
    }  
}  
  
return answer;  
}
```

Alternative with upper mid:

```
int binarySearchMaxAnswer(int left, int right) {  
    while (left < right) {  
        int mid = left + (right - left + 1) / 2;  
  
        if (can(mid)) {  
            left = mid;  
        } else {  
            right = mid - 1;  
        }  
    }  
  
    return left;  
}
```

Important:

Use upper mid to avoid infinite loop.

Template 3 — Minimize Maximum

```
int left = minimumPossible;  
int right = maximumPossible;  
  
while (left < right) {  
    int mid = left + (right - left) / 2;  
  
    if (canKeepMaxAtMost(mid)) {  
        right = mid;  
    } else {  
        left = mid + 1;  
    }  
}  
  
return left;
```

Template 4 — Maximize Minimum

```

int left = minimumPossible;
int right = maximumPossible;

while (left < right) {
    int mid = left + (right - left + 1) / 2;

    if (canMaintainAtLeast(mid)) {
        left = mid;
    } else {
        right = mid - 1;
    }
}

return left;

```

Template 5 — K-th Smallest by Counting

```

int left = minValue;
int right = maxValue;

while (left < right) {
    int mid = left + (right - left) / 2;

    if (countLessEqual(mid) >= k) {
        right = mid;
    } else {
        left = mid + 1;
    }
}

return left;

```

11. Major Variations

Minimum speed	first feasible speed	Koko Eating Bananas
Minimum capacity	first feasible capacity	Ship Packages
Smallest divisor	first feasible divisor	Smallest Divisor
Minimum days	first feasible day	Bouquets

Variation	Search Goal	Example
Minimize largest sum	first feasible max sum	Split Array
Maximize min distance	last feasible distance	Magnetic Force
Maximum allocation	last feasible candies	Maximum Candies
Minimum time	first feasible time	Minimum Time Trips
K-th smallest	first value with count $\geq k$	Multiplication Table
Minimize max penalty	first feasible penalty	Balls in a Bag

12. Full Problem Walkthrough 1 — LeetCode 875: Koko Eating Bananas

12.1 Problem Summary

Koko has banana piles.

She eats at speed k bananas per hour.

Each hour, she chooses one pile and eats up to k bananas from it.

Return minimum k so she can finish within h hours.

Example:

piles = {3, 6, 7, 11}

$h = 8$

Output:

4

12.2 How to Identify Binary Search on Answer

Problem asks:

- minimum speed
- if a speed works, any larger speed also works

Monotonic:

speed: 1 2 3 4 5 ...

works: F F F T T ...

Find first true.

12.3 Search Space

Minimum speed:

1

Maximum speed:

max(piles)

Eating faster than largest pile is unnecessary.

12.4 Feasibility Function

For speed k, each pile takes:

ceil(pile / k)

hours.

Total hours must be:

$\leq h$

12.5 C++ Code

```
class Solution {
public:
    bool canFinish(vector<int>& piles, int h, int speed) {
        long long hours = 0;

        for (int pile : piles) {
            hours += (pile + speed - 1LL) / speed;

            if (hours > h) {
                return false;
            }
        }

        return true;
    }

    int minEatingSpeed(vector<int>& piles, int h) {
        int left = 1;
        int right = *max_element(piles.begin(), piles.end());

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (canFinish(piles, h, mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
    }
};
```

```

    }

    return left;
}
};

```

12.6 Dry Run

piles = {3,6,7,11}
h = 8

Search range:

1 to 11

6	$1+1+2+2 = 6$	yes
3	$1+2+3+4 = 10$	no
5	$1+2+2+3 = 8$	yes
4	$1+2+2+3 = 8$	yes

Minimum working speed = 4.

12.7 Complexity

Let:

- n = number of piles
- M = max pile

Each feasibility check: $O(n)$

Binary search range: $O(\log M)$

Total:

$O(n \log M)$

Space: $O(1)$

12.8 Edge Cases

- h equals number of piles
- one pile
- all piles same
- very large pile values
- speed 1 works

12.9 Common Mistakes

Mistake 1: Using integer division directly

Wrong:

hours += pile / speed;

Correct:

hours += (pile + speed - 1) / speed;

[Mistake 2: Wrong search upper bound](#)

Use max pile, not sum.

12.10 Similar Problems

- Smallest Divisor
- Minimum Speed to Arrive on Time
- Minimum Time to Complete Trips

13. Full Problem Walkthrough 2 — LeetCode 1011: Capacity To Ship Packages Within D Days

13.1 Problem Summary

Given package weights in order, ship all packages within days.

Each day, ship packages in order until capacity would be exceeded.

Return minimum ship capacity.

Example:

weights = {1,2,3,4,5,6,7,8,9,10}

days = 5

Output:

15

13.2 How to Identify Binary Search on Answer

Problem asks:

- minimum capacity
- if capacity works, larger capacity also works
- packages must remain in order

Monotonic:

capacity too small -> impossible

capacity large enough -> possible

Find first feasible capacity.

13.3 Search Space

Minimum capacity:

max(weights)

Because capacity must carry largest package.

Maximum capacity:

```
sum(weights)
```

Because one day can carry all.

13.4 Feasibility Function

Given capacity, count days needed.

Start with one day.

Add packages in order.

If current day exceeds capacity, start new day.

Works if days needed $\leq$ given days.

13.5 C++ Code

```
class Solution {
public:
    bool canShip(vector<int>& weights, int days, int capacity) {
        int usedDays = 1;
        int currentLoad = 0;

        for (int w : weights) {
            if (currentLoad + w > capacity) {
                usedDays++;
                currentLoad = 0;
            }

            currentLoad += w;

            if (usedDays > days) {
                return false;
            }
        }

        return true;
    }

    int shipWithinDays(vector<int>& weights, int days) {
        int left = *max_element(weights.begin(), weights.end());
        int right = accumulate(weights.begin(), weights.end(), 0);

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (canShip(weights, days, mid)) {
                right = mid;
            }
        }

        return left;
    }
};
```

```
        } else {  
            left = mid + 1;  
        }  
    }  
  
    return left;  
}  
};
```

13.6 Dry Run Concept

Capacity too low means too many days.
Capacity high enough means days fit.
So if mid works, try smaller capacity.

13.7 Complexity

Let:

- n = number of weights
- S = sum of weights

Time:

$O(n \log S)$

Space: $O(1)$

13.8 Edge Cases

- days = 1, answer is sum
- days = weights.size(), answer is max weight
- one package
- increasing weights

13.9 Common Mistakes

Mistake 1: Lower bound as 1

This wastes time and can break if capacity less than max package is not handled.
Correct:

left = max(weights)

Mistake 2: Reordering packages

Packages must stay in original order.

14. Full Problem Walkthrough 3 — LeetCode 1283: Find the Smallest Divisor Given a Threshold

14.1 Problem Summary

Given array nums and threshold.

Choose divisor d.

Compute:

```
sum(ceil(nums[i] / d))
```

Return smallest divisor such that sum $\leq$ threshold.

Example:

```
nums = {1,2,5,9}
```

```
threshold = 6
```

Output:

```
5
```

14.2 How to Identify Binary Search on Answer

Problem asks:

- smallest divisor
- larger divisor gives smaller/equal sum
- need sum $\leq$ threshold

Monotonic:

divisor small $\rightarrow$ sum too large

divisor large $\rightarrow$ works

Find first true.

14.3 Search Space

Minimum divisor:

```
1
```

Maximum divisor:

```
max(nums)
```

14.4 C++ Code

```
class Solution {
public:
    bool can(vector<int>& nums, int threshold, int divisor) {
        int sum = 0;

        for (int x : nums) {
            sum += (x + divisor - 1) / divisor;

            if (sum > threshold) {
                return false;
            }
        }
    }
}
```

```

        return true;
    }

    int smallestDivisor(vector<int>& nums, int threshold) {
        int left = 1;
        int right = *max_element(nums.begin(), nums.end());

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (can(nums, threshold, mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

14.5 Complexity

Time:

$O(n \log M)$

where M = max value.

Space: $O(1)$

14.6 Similar Problems

- Koko Eating Bananas
- Minimum Speed to Arrive on Time

15. Full Problem Walkthrough 4 — LeetCode 1482: Minimum Number of Days to Make m Bouquets

15.1 Problem Summary

Given bloomDay, integer m, and integer k.

Need make m bouquets.

Each bouquet needs k adjacent flowers.

Flower i blooms on $\text{bloomDay}[i]$.
Return minimum day to make bouquets, or -1 if impossible.
Example:

```
bloomDay = {1,10,3,10,2}
m = 3
k = 1
```

Output:

3

15.2 How to Identify Binary Search on Answer

Problem asks:

- minimum day
- if day D works, any later day also works
- feasibility can be checked greedily

Find first feasible day.

15.3 Search Space

Minimum day:

```
min(bloomDay)
```

Maximum day:

```
max(bloomDay)
```

Before search, check impossible:

```
m * k > bloomDay.size()
```

Use long long.

15.4 Feasibility Function

For a given day:

- scan flowers
- count consecutive bloomed flowers
- whenever consecutive count reaches k , make bouquet and reset count

15.5 C++ Code

```
class Solution {
public:
    bool canMake(vector<int>& bloomDay, int m, int k, int day) {
        int bouquets = 0;
        int consecutive = 0;

        for (int bloom : bloomDay) {
            if (bloom <= day) {
                consecutive++;
            }
        }
    }
};
```

```

        if (consecutive == k) {
            bouquets++;
            consecutive = 0;

            if (bouquets >= m) {
                return true;
            }
        } else {
            consecutive = 0;
        }
    }

    return bouquets >= m;
}

int minDays(vector<int>& bloomDay, int m, int k) {
    if (1LL * m * k > bloomDay.size()) {
        return -1;
    }

    int left = *min_element(bloomDay.begin(), bloomDay.end());
    int right = *max_element(bloomDay.begin(), bloomDay.end());

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (canMake(bloomDay, m, k, mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}
};

```

15.6 Dry Run Concept

For day 3:

```
bloomDay = {1,10,3,10,2}
```

Flowers bloomed:

yes, no, yes, no, yes

With $k = 1$, can make 3 bouquets.

Answer is 3.

15.7 Complexity

Let:

- n = number of flowers
- $D = \text{maxDay} - \text{minDay}$

Time:

$O(n \log D)$

Space: $O(1)$

15.8 Edge Cases

- impossible due to not enough flowers
- $k = 1$
- $m = 1$
- all bloom same day
- adjacent requirement

15.9 Common Mistakes

Mistake 1: Ignoring adjacency

You cannot just count bloomed flowers.

Need consecutive groups.

Mistake 2: Overflow in $m * k$

Use:

$1LL * m * k$

16. Full Problem Walkthrough 5 — LeetCode 410: Split Array Largest Sum

16.1 Problem Summary

Given array `nums` and integer `k`, split array into `k` non-empty contiguous subarrays.

Minimize the largest subarray sum.

Example:

`nums = {7,2,5,10,8}`

`k = 2`

Output:

18

Split:

{7,2,5}, {10,8}

Largest sum = 18.

16.2 How to Identify Binary Search on Answer

Problem asks:

- minimize largest subarray sum
- contiguous groups
- can check if max allowed sum works

This is classic minimize maximum.

16.3 Search Space

Smallest possible largest sum:

max(nums)

Largest possible largest sum:

sum(nums)

16.4 Feasibility Function

Given max allowed sum limit, count how many subarrays are needed.

Greedily pack numbers until adding next exceeds limit.

If needed groups $\leq k$, limit works.

16.5 C++ Code

```
class Solution {
public:
    bool canSplit(vector<int>& nums, int k, int limit) {
        int groups = 1;
        int currentSum = 0;

        for (int x : nums) {
            if (currentSum + x > limit) {
                groups++;
                currentSum = 0;
            }

            currentSum += x;
        }

        if (groups > k) {
            return false;
        }
    }
}
```

```

        return true;
    }

    int splitArray(vector<int>& nums, int k) {
        int left = *max_element(nums.begin(), nums.end());
        int right = accumulate(nums.begin(), nums.end(), 0);

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (canSplit(nums, k, mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

16.6 Why Greedy Feasibility Works

For a fixed limit, to minimize number of groups, put as many elements as possible in current group without exceeding limit.

If even this greedy method needs more than k groups, no other split can do better.

16.7 Complexity

Let:

- n = nums size
- S = sum(nums)

Time:

$O(n \log S)$

Space: $O(1)$

16.8 Similar Problems

- Ship Packages Within D Days
- Painter's Partition
- Allocate Books

17. Full Problem Walkthrough 6 — LeetCode 1552: Magnetic Force Between Two Balls

17.1 Problem Summary

Given basket positions and m balls.

Place balls in baskets to maximize the minimum distance between any two balls.

Example:

position = {1,2,3,4,7}
 $m = 3$

Output:

3

Place at:

1, 4, 7

Minimum distance = 3.

17.2 How to Identify Binary Search on Answer

Problem asks:

- maximize minimum distance
- if distance d works, smaller distances also work
- if d does not work, larger distances do not work

Pattern:

true true true false false

Find last true.

17.3 Search Space

Sort positions.

Minimum distance:

1

Maximum distance:

maxPosition - minPosition

17.4 Feasibility Function

Can we place m balls such that each chosen position is at least $dist$ apart?

Greedy:

- place first ball at first position
- place next ball at earliest position at least $dist$ away
- count placed balls

17.5 C++ Code

```
class Solution {
public:
    bool canPlace(vector<int>& position, int m, int dist) {
        int placed = 1;
        int last = position[0];

        for (int i = 1; i < position.size(); i++) {
            if (position[i] - last >= dist) {
                placed++;
                last = position[i];

                if (placed >= m) {
                    return true;
                }
            }
        }

        return false;
    }

    int maxDistance(vector<int>& position, int m) {
        sort(position.begin(), position.end());

        int left = 1;
        int right = position.back() - position.front();

        while (left < right) {
            int mid = left + (right - left + 1) / 2;

            if (canPlace(position, m, mid)) {
                left = mid;
            } else {
                right = mid - 1;
            }
        }

        return left;
    }
};
```

17.6 Why Upper Mid?

We are finding maximum feasible value.

Use:

```
mid = left + (right - left + 1) / 2
```

so when left and right are adjacent, mid becomes right, preventing infinite loop.

17.7 Complexity

Sorting:

```
O(n log n)
```

Binary search:

```
O(n log range)
```

Total:

```
O(n log n + n log range)
```

Space: $O(1)$ extra.

17.8 Similar Problems

- Aggressive Cows
- Minimize Max Distance to Gas Station
- Kth Smallest Pair Distance

18. Full Problem Walkthrough 7 — LeetCode 2226: Maximum Candies Allocated to K Children

18.1 Problem Summary

You have candy piles.

You can split piles, but cannot merge piles.

Each child must get exactly the same number of candies.

Return maximum candies each child can get.

Example:

```
candies = {5, 8, 6}
```

```
k = 3
```

Output:

```
5
```

18.2 How to Identify Binary Search on Answer

Problem asks:

- maximum candies per child
- if x candies per child works, any smaller amount works

- if x does not work, larger amount does not work
- Find last feasible value.
-

18.3 Search Space

Minimum:

0

Maximum:

max(candies)

If total candies < k, answer is 0.

18.4 Feasibility Function

For guess x, each pile gives:

pile / x

children.

If total children >= k, x works.

18.5 C++ Code

```
class Solution {
public:
    bool canGive(vector<int>& candies, long long k, int each) {
        long long children = 0;

        for (int pile : candies) {
            children += pile / each;

            if (children >= k) {
                return true;
            }
        }

        return false;
    }

    int maximumCandies(vector<int>& candies, long long k) {
        long long total = accumulate(candies.begin(), candies.end(), 0LL);

        if (total < k) {
            return 0;
        }

        int left = 1;
```

```
int right = *max_element(candies.begin(), candies.end());

while (left < right) {
    int mid = left + (right - left + 1) / 2;

    if (canGive(candies, k, mid)) {
        left = mid;
    } else {
        right = mid - 1;
    }
}

return left;
};
```

18.6 Why Not Start Left at 0?

Division by zero.

We handle impossible case first.

Then search from 1.

18.7 Complexity

Time:

$O(n \log M)$

Space: $O(1)$

18.8 Similar Problems

- Magnetic Force Between Two Balls
 - Koko Eating Bananas
 - Maximum minimum allocation problems
-

19. Full Problem Walkthrough 8 — LeetCode 2187: Minimum Time to Complete Trips

19.1 Problem Summary

Each bus takes $\text{time}[i]$ to complete one trip.

All buses can work simultaneously.

Return minimum time needed to complete at least totalTrips .

Example:

time = {1,2,3}
totalTrips = 5

Output:

3

19.2 How to Identify Binary Search on Answer

Problem asks:

- minimum time
- if time T can complete enough trips, any larger time also works

Find first feasible time.

19.3 Search Space

Minimum time:

1

Maximum time:

min(time) * totalTrips

Fastest bus doing all trips alone.

Use long long.

19.4 Feasibility Function

At time T, bus i completes:

$T / \text{time}[i]$

trips.

Sum trips.

If sum $\geq$ totalTrips, feasible.

19.5 C++ Code

```
class Solution {
public:
    bool canComplete(vector<int>& time, long long totalTrips, long long
givenTime) {
        long long trips = 0;

        for (int t : time) {
            trips += givenTime / t;

            if (trips >= totalTrips) {
                return true;
            }
        }
    }
}
```

```

        return false;
    }

    long long minimumTime(vector<int>& time, int totalTrips) {
        long long left = 1;
        long long fastest = *min_element(time.begin(), time.end());
        long long right = fastest * totalTrips;

        while (left < right) {
            long long mid = left + (right - left) / 2;

            if (canComplete(time, totalTrips, mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

19.6 Complexity

Time:

$O(n \log \text{answer})$

Space: $O(1)$

19.7 Edge Cases

- one bus
- huge totalTrips
- all buses same speed
- answer exceeds int range

20. Full Problem Walkthrough 9 — LeetCode 1870: Minimum Speed to Arrive on Time

20.1 Problem Summary

You travel through train rides with distances `dist`.

For every ride except the last, arrival time is rounded up to next integer hour.
Return minimum integer speed to arrive within hour.
If impossible, return -1.

20.2 How to Identify Binary Search on Answer

Problem asks:

- minimum speed
- if speed works, faster speed also works

Find first feasible speed.

20.3 Important Impossible Condition

If there are n rides, first $n - 1$ rides each take at least 1 integer hour.
So if:

hour $\leq n - 1$

may be impossible, except constraints with decimals need careful comparison.
Usually:

if (hour $\leq n - 1$) return -1;

because last ride needs positive time.

20.4 Feasibility

For speed v :

For all rides except last:

$\text{ceil}(\text{dist}[i] / v)$

Last ride:

$\text{dist}[n-1] / v$

as double.

20.5 C++ Code

```
class Solution {
public:
    bool canArrive(vector<int>& dist, double hour, int speed) {
        double time = 0.0;
        int n = dist.size();

        for (int i = 0; i < n; i++) {
            double rideTime = (double)dist[i] / speed;

            if (i == n - 1) {
                time += rideTime;
            } else {
                time += ceil(rideTime);
            }
        }
    }
};
```

```

    }

    if (time > hour) {
        return false;
    }
}

return time <= hour;
}

int minSpeedOnTime(vector<int>& dist, double hour) {
    int n = dist.size();

    if (hour <= n - 1) {
        return -1;
    }

    int left = 1;
    int right = 10000000;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (canArrive(dist, hour, mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return canArrive(dist, hour, left) ? left : -1;
}
};

```

20.6 Complexity

Time:

$O(n \log 10^7)$

Space: $O(1)$

20.7 Common Mistakes

- rounding last ride
- using integer division for time

- wrong impossible condition
 - too small upper bound
-

21. Full Problem Walkthrough 10 — LeetCode 1760: Minimum Limit of Balls in a Bag

21.1 Problem Summary

You have bags with balls.

In one operation, split one bag into two bags.

You can do at most `maxOperations`.

Minimize maximum number of balls in any bag.

Example:

```
nums = {9}
maxOperations = 2
```

Output:

```
3
```

Split 9 into 3,3,3.

21.2 How to Identify Binary Search on Answer

Problem asks:

- minimize maximum bag size
- if max size X is achievable, larger sizes are also achievable

Find first feasible penalty.

21.3 Feasibility Function

For a bag with balls, to make all resulting bags have at most limit, operations needed:

```
(balls - 1) / limit
```

Why?

Example:

```
balls = 9
limit = 3
```

Need split into 3 bags.

Operations = 2.

Formula:

```
(9 - 1) / 3 = 2
```

21.4 C++ Code

```

class Solution {
public:
    bool can(vector<int>& nums, int maxOperations, int limit) {
        long long operations = 0;

        for (int balls : nums) {
            operations += (balls - 1) / limit;

            if (operations > maxOperations) {
                return false;
            }
        }

        return true;
    }

    int minimumSize(vector<int>& nums, int maxOperations) {
        int left = 1;
        int right = *max_element(nums.begin(), nums.end());

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (can(nums, maxOperations, mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

21.5 Complexity

Time:

$O(n \log M)$

Space: $O(1)$

21.6 Common Mistakes

Using:

$\text{balls} / \text{limit}$

Wrong when balls divides evenly.

Correct:

$(\text{balls} - 1) / \text{limit}$

22. Full Problem Walkthrough 11 — LeetCode 774: Minimize Max Distance to Gas Station

Premium-style but important.

22.1 Problem Summary

Given sorted gas station positions and integer k , add k new stations.

Minimize the maximum distance between adjacent stations.

22.2 How to Identify Binary Search on Answer

Answer is a real number.

If max gap d is achievable, any larger gap is also achievable.

Find minimum feasible distance.

22.3 Feasibility

For each gap length d , number of stations needed so every segment $\leq d$:

$\text{ceil}(\text{gap} / d) - 1$

If total needed $\leq k$, feasible.

22.4 C++ Code

```
double minmaxGasDist(vector<int>& stations, int k) {  
    double left = 0.0;  
    double right = stations.back() - stations.front();  
  
    auto can = [&](double dist) {  
        int needed = 0;  
  
        for (int i = 1; i < stations.size(); i++) {  
            double gap = stations[i] - stations[i - 1];  
  
            needed += (int)ceil(gap / dist) - 1;  
  
            if (needed > k) {  
                return false;  
            }  
        }  
        return true;  
    };  
  
    while (left < right) {  
        double mid = (left + right) / 2;  
        if (can(mid)) {  
            right = mid;  
        } else {  
            left = mid;  
        }  
    }  
    return left;  
}
```

```

    }
}

return true;
};

for (int iter = 0; iter < 60; iter++) {
    double mid = (left + right) / 2.0;

    if (can(mid)) {
        right = mid;
    } else {
        left = mid;
    }
}

return right;
}

```

22.5 Why Fixed Iterations?

For floating-point binary search, avoid relying on exact equality.
60 iterations gives enough precision.

22.6 Complexity

Time:

$O(n * 60)$

Space: $O(1)$

22.7 Common Mistakes

- integer binary search on real answer
 - precision issues
 - wrong station-needed formula
-

23. Full Problem Walkthrough 12 — LeetCode 668: Kth Smallest Number in Multiplication Table

23.1 Problem Summary

Given multiplication table of size $m \times n$, return k -th smallest number.

Example:

m = 3
n = 3
k = 5

Table:

1 2 3
2 4 6
3 6 9

5th smallest is 3.

23.2 How to Identify Binary Search on Answer

Need k-th smallest value.

Search value range:

1 to m * n

For guessed value x, count how many table values are $\leq x$.

If count $\geq k$, answer is $\leq x$.

Find first value with count $\geq k$.

23.3 Count Function

In row i, values are:

i, 2i, 3i, ..., n*i

Count $\leq x$:

$\min(n, x / i)$

23.4 C++ Code

```
class Solution {
public:
    int countLessEqual(int m, int n, int x) {
        int count = 0;

        for (int i = 1; i <= m; i++) {
            count += min(n, x / i);
        }

        return count;
    }

    int findKthNumber(int m, int n, int k) {
        int left = 1;
        int right = m * n;
```

```
while (left < right) {
    int mid = left + (right - left) / 2;

    if (countLessEqual(m, n, mid) >= k) {
        right = mid;
    } else {
        left = mid + 1;
    }
}

return left;
}
```

23.5 Complexity

Time:

$O(m \log(mn))$

Can optimize by iterating smaller dimension.

Space: $O(1)$

23.6 Similar Problems

- Kth Smallest Element in Sorted Matrix
- Find K-th Smallest Pair Distance
- Kth Smallest Prime Fraction

24. Full Problem Walkthrough 13 — LeetCode 719: Find K-th Smallest Pair Distance

24.1 Problem Summary

Given array `nums`, define distance of pair (i, j) as:

`abs(nums[i] - nums[j])`

Return k-th smallest pair distance.

Example:

`nums = {1,3,1}`

`k = 1`

Output:

0

24.2 How to Identify Binary Search on Answer

Need k-th smallest distance.

Search possible distance:

0 to $\max(\text{nums}) - \min(\text{nums})$

For guessed distance d , count how many pairs have distance $\leq d$.

If count $\geq k$, answer $\leq d$.

Find first distance with count $\geq k$.

24.3 Count Pairs $\leq$ Distance

Sort nums.

Use sliding window:

For each right, move left while:

$\text{nums}[\text{right}] - \text{nums}[\text{left}] > d$

Then pairs ending at right:

$\text{right} - \text{left}$

24.4 C++ Code

```
class Solution {
public:
    int countPairs(vector<int>& nums, int dist) {
        int left = 0;
        int count = 0;

        for (int right = 0; right < nums.size(); right++) {
            while (nums[right] - nums[left] > dist) {
                left++;
            }

            count += right - left;
        }

        return count;
    }

    int smallestDistancePair(vector<int>& nums, int k) {
        sort(nums.begin(), nums.end());

        int left = 0;
        int right = nums.back() - nums.front();
```

```
while (left < right) {
    int mid = left + (right - left) / 2;

    if (countPairs(nums, mid) >= k) {
        right = mid;
    } else {
        left = mid + 1;
    }
}

return left;
}
```

24.5 Complexity

Sorting:

$O(n \log n)$

Each count:

$O(n)$

Binary search over distance:

$O(\log W)$

Total:

$O(n \log n + n \log W)$

24.6 Similar Problems

- Kth Smallest Number in Multiplication Table
- Kth Smallest Element in Sorted Matrix
- Count pairs with distance $\leq x$

25. Full Problem Walkthrough 14 — LeetCode 378: Kth Smallest Element in a Sorted Matrix

25.1 Problem Summary

Given $n \times n$ matrix where each row and column is sorted ascending, return k -th smallest element.

Example:

```
matrix = {  
    {1, 5, 9},  
    {10, 11, 13},  
    {12, 13, 15}  
}  
k = 8
```

Output:

13

25.2 How to Identify Binary Search on Answer

Need k-th smallest.

Value range:

matrix[0][0] to matrix[n-1][n-1]

For guessed value x, count how many elements are $\leq x$.

Find first value with count $\geq k$.

25.3 Count Less Equal

Start bottom-left.

If current value $\leq x$, then everything above in this column is $\leq x$.

Add row + 1, move right.

If current value $> x$, move up.

25.4 C++ Code

```
class Solution {  
public:  
    int countLessEqual(vector<vector<int>>& matrix, int x) {  
        int n = matrix.size();  
  
        int row = n - 1;  
        int col = 0;  
        int count = 0;  
  
        while (row >= 0 && col < n) {  
            if (matrix[row][col] <= x) {  
                count += row + 1;  
                col++;  
            } else {  
                row--;  
            }  
        }  
    }  
}
```

```

        return count;
    }

    int kthSmallest(vector<vector<int>>& matrix, int k) {
        int n = matrix.size();

        int left = matrix[0][0];
        int right = matrix[n - 1][n - 1];

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (countLessEqual(matrix, mid) >= k) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

25.5 Complexity

Each count:

$O(n)$

Binary search value range:

$O(\log(\text{max-min}))$

Total:

$O(n \log \text{range})$

Space: $O(1)$

25.6 Alternatives

Heap approach:

$O(k \log n)$

Binary search is good when value range is manageable.

26. Full Problem Walkthrough 15 — LeetCode 287: Find the Duplicate Number

26.1 Problem Summary

Given array nums containing $n + 1$ integers where each integer is in range 1 to n . There is exactly one repeated number.

Return duplicate without modifying array and using $O(1)$ extra space.

Example:

```
nums = {1,3,4,2,2}
```

Output:

```
2
```

26.2 Binary Search on Value

This problem is not sorted.

But answer is in value range:

```
1 to n
```

For guessed value mid, count numbers $\leq$ mid.

If count $>$ mid, duplicate must be in lower half.

Why?

In range $1..mid$, there should be at most mid numbers if no duplicate exists there.

26.3 C++ Code

```
class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int left = 1;
        int right = nums.size() - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            int count = 0;

            for (int x : nums) {
                if (x <= mid) {
                    count++;
                }
            }
        }
    }
};
```

```
        if (count > mid) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}
```

26.4 Complexity

Time:

$O(n \log n)$

Space: $O(1)$

26.5 Alternative

Floyd's cycle detection solves in $O(n)$ time and $O(1)$ space.
But value-space binary search is a useful pattern.

27. Mini Problem 1 — Minimized Maximum of Products Distributed to Stores

Pattern:

Minimize maximum products per store.

Search answer from 1 to max quantity.

Feasibility:

$\text{storesNeeded} += \text{ceil}(\text{quantity} / \text{maxPerStore})$

If $\text{storesNeeded} \leq n$, feasible.

28. Mini Problem 2 — Allocate Books / Painter's Partition

Pattern:

Same as Split Array Largest Sum.

Search maximum pages/time.

Feasibility:

Count students/painters needed if each gets at most limit.

29. Mini Problem 3 — Aggressive Cows

Pattern:

Same as Magnetic Force.

Sort stall positions.

Search distance.

Feasibility:

Greedily place cows at earliest valid stall.

30. Mini Problem 4 — K-th Missing Positive Number

Can be solved by binary search.

For sorted array `arr`, missing count before index `i`:

$arr[i] - (i + 1)$

Find first index where missing count $\geq k$.

31. Common Edge Cases

31.1 Impossible Case

Examples:

Bouquets:

$m * k > n$

Return -1.

Minimum speed arrival:

not enough time.

31.2 Lower Bound Too Low

For shipping capacity, lower bound must be max weight.

For split array, lower bound must be max element.

31.3 Upper Bound Too Low

If upper bound does not include answer, binary search fails.

Always choose safe upper bound.

31.4 Overflow

Use long long for:

- sums
- products
- time

- total trips
 - mid calculations for large ranges
-

31.5 Division by Zero

If answer can be 0, do not use it as divisor.
Handle 0 separately.

31.6 Floating Answer

Use fixed iterations or epsilon.
Do not wait for exact equality.

31.7 Feasibility Direction

Know whether true means:

- try smaller
- try larger

Minimum feasible:

if can(mid): right = mid

Maximum feasible:

if can(mid): left = mid

32. Common Mistakes and Wrong Code

Mistake 1 — No Monotonicity Check

Before binary searching, prove:

if mid works, larger/smaller also works

Mistake 2 — Wrong Boundaries

Bad:

left = 0

for divisor problems.

Could divide by zero.

Mistake 3 — Wrong Feasibility Function

The binary search is only as correct as can(mid).

Mistake 4 — Infinite Loop in Maximum Search

Wrong:

mid = (left + right) / 2;
if (can(mid)) left = mid;

If $\text{left} + 1 == \text{right}$, mid equals left forever.

Correct:

$\text{mid} = \text{left} + (\text{right} - \text{left} + 1) / 2;$

Mistake 5 — Using Int for Large Answer

Minimum time/trips can exceed int.

Use long long.

Mistake 6 — Forgetting Contiguity

For Split Array / Shipping, groups must be contiguous and order must remain.

33. Interview Explanation Templates

33.1 Minimum Feasible

I binary search the answer because if a candidate value works, then any larger value also works. So the feasibility pattern is false then true. I search for the first feasible value.

33.2 Maximum Feasible

I binary search the answer because if a candidate value works, then any smaller value also works. So the feasibility pattern is true then false. I search for the last feasible value.

33.3 Feasibility Function

For each guessed answer, I simulate greedily to check if it satisfies the constraint. This check is linear, and binary search reduces the number of guesses logarithmically.

33.4 Minimize Maximum

The answer is the smallest maximum limit that allows the task to be completed. If a limit works, larger limits also work, so I binary search the minimum working limit.

33.5 Maximize Minimum

The answer is the largest minimum distance/value that can still be achieved. If a distance works, any smaller distance works, so I binary search the largest feasible distance.

34. Complexity Patterns

Koko Eating Bananas	$O(n)$	$O(n \log M)$

Problem Type	Feasibility	Total Time
Ship Packages	$O(n)$	$O(n \log S)$
Smallest Divisor	$O(n)$	$O(n \log M)$
Bouquets	$O(n)$	$O(n \log D)$
Split Array	$O(n)$	$O(n \log S)$
Magnetic Force	$O(n)$ after sort	$O(n \log n + n \log R)$
Maximum Candies	$O(n)$	$O(n \log M)$
Minimum Time Trips	$O(n)$	$O(n \log T)$
Kth Multiplication Table	$O(m)$	$O(m \log mn)$
Kth Pair Distance	$O(n)$ after sort	$O(n \log n + n \log W)$
Kth Sorted Matrix	$O(n)$	$O(n \log \text{range})$

35. Time-Boxed Practice Guidance

Intro Binary Search on Answer

Target:

20–30 minutes

Examples:

- Koko Eating Bananas
- Smallest Divisor
- Ship Packages

Core Medium Problems

Target:

30–45 minutes

Examples:

- Bouquets
- Split Array Largest Sum
- Magnetic Force
- Maximum Candies
- Minimum Time Trips

Harder K-th / Counting Problems

Target:

45–70 minutes

Examples:

- Kth Smallest Pair Distance
 - Kth Smallest Multiplication Table
 - Kth Smallest Sorted Matrix
 - Gas Station Distance
-

36. Expanded Practice Set

36.1 Must-Do Medium

- 1.1. Koko Eating Bananas
 - 2.2. Capacity To Ship Packages Within D Days
 - 3.3. Find the Smallest Divisor Given a Threshold
 - 4.4. Minimum Number of Days to Make m Bouquets
 - 5.5. Split Array Largest Sum
 - 6.6. Magnetic Force Between Two Balls
 - 7.7. Maximum Candies Allocated to K Children
 - 8.8. Minimum Time to Complete Trips
 - 9.9. Minimum Limit of Balls in a Bag
 - 10.10. Minimum Speed to Arrive on Time
-

36.2 Strong Interview Practice

- 11.11. Minimize Max Distance to Gas Station
 - 12.12. Kth Smallest Number in Multiplication Table
 - 13.13. Find K-th Smallest Pair Distance
 - 14.14. Kth Smallest Element in a Sorted Matrix
 - 15.15. Find the Duplicate Number
 - 16.16. Maximum Number of Removable Characters
 - 17.17. Minimized Maximum of Products Distributed to Any Store
 - 18.18. Ugly Number III
 - 19.19. Maximum Tastiness of Candy Basket
 - 20.20. Maximum Running Time of N Computers
-

36.3 Hard / Stretch

- 21.21. Find K-th Smallest Pair Distance
 - 22.22. Kth Smallest Number in Multiplication Table
 - 23.23. K-th Smallest Prime Fraction
 - 24.24. Nth Magical Number
 - 25.25. Ugly Number III
 - 26.26. Maximum Running Time of N Computers
 - 27.27. Minimize Max Distance to Gas Station
-

37. Suggested Study Order

Step 1 — Minimum Feasible

12. Koko Eating Bananas
13. Smallest Divisor
14. Ship Packages
15. Bouquets

Goal:

Learn first true binary search.

Step 2 — Minimize Maximum

- 16. Split Array Largest Sum
- 17. Minimum Limit of Balls in a Bag
- 18. Minimized Maximum Products

Goal:

Learn max allowed value feasibility.

Step 3 — Maximum Feasible

- 19. Magnetic Force Between Two Balls
- 20. Maximum Candies Allocated to K Children
- 21. Maximum Tastiness of Candy Basket

Goal:

Learn last true binary search with upper mid.

Step 4 — Minimum Time / Speed

- 22. Minimum Time to Complete Trips
- 23. Minimum Speed to Arrive on Time

Goal:

Learn large bounds and long long.

Step 5 — K-th Smallest by Counting

- 24. Kth Smallest Number in Multiplication Table
- 25. Kth Smallest Pair Distance
- 26. Kth Smallest Element in Sorted Matrix

Goal:

Learn count $\leq$ mid.

38. Pattern Decision Table

minimum speed	first true	time $\leq$ limit
minimum capacity	first true	days $\leq$ limit
smallest divisor	first true	sum $\leq$ threshold
minimum day	first true	bouquets $\geq$ m
minimize largest sum	first true	groups $\leq$ k
maximize minimum distance	last true	can place all
maximum candies per child	last true	children $\geq$ k
minimum time	first true	trips $\geq$ total

Problem Clue	Search Type	Feasibility
minimize max bag size	first true	operations <= maxOps
k-th smallest	first true	count <= mid >= k
duplicate value search	first true	count <= mid > mid

39. Final Cheat Sheet

Minimum Feasible

```

while (left < right) {
    int mid = left + (right - left) / 2;

    if (can(mid)) {
        right = mid;
    } else {
        left = mid + 1;
    }
}

return left;

```

Maximum Feasible

```

while (left < right) {
    int mid = left + (right - left + 1) / 2;

    if (can(mid)) {
        left = mid;
    } else {
        right = mid - 1;
    }
}

return left;

```

Ceiling Division

```
int ceilDiv = (a + b - 1) / b;
```

With long long:

```
long long ceilDiv = (a + b - 1LL) / b;
```

Minimize Maximum Boundaries

```
left = maxElement;  
right = sum;
```

Used for:

- Ship Packages
- Split Array
- Allocate Books

Maximize Minimum Distance Boundaries

```
sort(position.begin(), position.end());
```

```
left = 1;  
right = position.back() - position.front();
```

K-th Smallest Count Pattern

```
while (left < right) {  
    int mid = left + (right - left) / 2;  
  
    if (countLessEqual(mid) >= k) {  
        right = mid;  
    } else {  
        left = mid + 1;  
    }  
}  
  
return left;
```

40. What to Master Before Chapter 11

Before moving to Quickselect, make sure you can:

- Identify numeric answer space
- Prove monotonic feasibility
- Choose correct lower and upper bounds
- Write can(mid) clearly
- Know first true vs last true
- Use upper mid for maximum feasible
- Use ceiling division
- Use long long for large values
- Solve Koko Eating Bananas
- Solve Ship Packages

- Solve Split Array Largest Sum
 - Solve Magnetic Force
 - Solve Minimum Time Trips
 - Solve K-th smallest by counting
 - Explain binary search on answer in interviews
-

41. Core Takeaway

Binary Search on Answer is not about searching an array.
It is about searching possible answers.

The core question is:

If I guess an answer, can I check whether it works?

Then ask:

Is that check monotonic?

If yes, binary search applies.

Minimum feasible problems look like:

false false false true true true

Maximum feasible problems look like:

true true true false false false

The most important skill is writing the feasibility function.

Binary search only narrows guesses.

The can(mid) function proves whether a guess is valid.
